

AI Trust Receipt Technical Specification

Candidate version 0.1.1 patch · Reference implementation profile

Emotional Infrastructure project

19 July 2026

Contents

Status and claim boundary	1
1. Purpose	2
2. Scope	2
3. Terms	2
4. System model	3
5. Data contracts and schema use	3
6. Trigger and consequence classification	4
6.1 Receipt trigger	4
6.2 Consequence classes	4
7. Pre-execution gate	4
7.1 Inputs	4
7.2 Deterministic rules	4
7.3 Temporal semantics	5
7.4 Failure containment	5
8. Receipt semantics	5
8.1 Required record	5
8.2 Event status	6
8.3 Identifier and ordering rules	6
9. Canonicalization and integrity	6
9.1 Reference digest profile	6
9.2 Verification	7
9.3 Storage	7
10. Human and machine consistency	7
11. Privacy, minimization, and excluded content	7
12. Notice, remedy, and affected-party access	8
13. Delegation	8
14. R1–R12 conformance profile	8
14.1 Requirements	8
14.2 Per-requirement values	8
14.3 Overall decision	9
14.4 Execution mode	9
15. Interfaces	9
16. Error handling and observability	9
17. Security considerations	10
18. Reference test suite	10
19. Versioning and extension	10
20. References	11

Status and claim boundary

Document status: Public-interest technical proposal

Patch status: v0.1.1 updates the reference implementation and machine-readable schemas for receipt identifier generation, configurable generated links, explicit reversibility, explicit protected-third-party-information handling, human-review evidence traceability, restricted cross-runtime digest parity, and expanded tests. The workbook remains a v0.1 candidate artifact.

This document defines a candidate technical specification for producing and assessing AI Trust Receipts. It converts the governance requirements in the AI Trust Receipt Workbook into testable data contracts, deterministic decision rules, lifecycle behavior, and conformance evidence.

This specification and its reference implementation are proposed materials. They are not an adopted standard, an accreditation or certification program, a legal-compliance determination, an external audit, regulator approval, or a guarantee that an action was fair, lawful, safe, or correct. A receipt records what an accountable system says occurred and the evidence by which that statement can be checked. Trustworthiness still depends on truthful inputs, sound governance, operational controls, secure implementation, and independent scrutiny where warranted.

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHOULD**, **SHOULD NOT**, and **MAY** in this document express the normative strength of candidate requirements. Because this is a candidate rather than an adopted standard, these words govern conformance to this package only.

1. Purpose

An AI-mediated action can cross organizational and technical boundaries faster than an affected person can understand, contest, or reverse it. A Trust Receipt addresses that asymmetry by coupling consequential action with a durable, minimally sufficient event record.

The specification has five technical objectives:

1. Stop an action before execution when authority, required evidence, receipt capability, or remedy capability is missing.
2. Produce a machine-verifiable event record for executed and non-executed outcomes.
3. Generate human-readable disclosure from the same canonical event record.
4. Apply a noncompensatory R1–R12 conformance decision, so strength in one requirement cannot cancel a failure in another.
5. Provide reproducible positive and negative test vectors for implementers and assessors.

2. Scope

This profile applies to an AI agent or AI-mediated service that attempts an externally meaningful action on behalf of a person or organization. It is especially relevant when an action persists beyond the immediate interaction; changes access, eligibility, financial position, identity, communication, or another material state; delegates authority; affects a person other than the requester; or requires a route to correction or appeal.

This profile does not prescribe a model architecture, orchestration framework, identity provider, database, public-key infrastructure, legal retention period, sector-specific consequence taxonomy, or user-interface design. Deployments **MUST** bind those elements in a documented deployment profile.

The package includes a restricted SHA-256 integrity demonstration. It does not provide issuer authentication, key custody, revocation, or non-repudiation. A production deployment that requires issuer-authenticated receipts **SHOULD** select an approved signature and key-management profile and state its trust assumptions.

3. Terms

Accountable organization means the legal or organizational entity that accepts responsibility for operating the agent and the review process.

Action request means the proposed action, target, state transition, consequence classification, confirmation state, human-review state, evidence identifiers actually viewed during a recorded review, reversibility declaration where known, and protected-third-party-information declaration where applicable submitted to the pre-execution gate.

Affected party means a person or organization whose interests, access, status, property, communication, or data are materially affected by the action, whether or not that party initiated the request.

Authority grant means the bounded authorization that identifies the granting party, grantee, objective, allowed and excluded actions, target scope, consequence ceiling, confirmation threshold, delegation rule, and validity interval.

Canonical event record means the single structured object from which the machine receipt, human projection, integrity value, and downstream verification are derived.

Consequence class means the declared impact level C0, C1, C2, or C3 defined in Section 6.

Material evidence means an input whose presence, provenance, status, or freshness can change whether the action is authorized or justified.

Receipt means a durable record of an attempted, executed, denied, delegated, escalated, reversed, or expired event state.

Remedy means the operational ability to contest, review, correct, reverse, restore, or otherwise address an action and its effects.

4. System model

The normative processing boundary contains five logical functions:

Function	Required responsibility
Request assembler	Construct a schema-valid action request without inventing authority or evidence.
Pre-execution gate	Evaluate authority, consequence, confirmation, delegation, evidence, receipt availability, and remedy availability.
Action executor	Apply the state change only after an allow decision and return the observed outcome.
Receipt service	Create a canonical event record, apply integrity protection, persist it, and expose verification and remedy.
Human renderer	Produce disclosure from the canonical receipt without an independent factual data path.

An implementation MAY combine these functions in one process. Their decision boundaries MUST remain distinguishable in logs and tests.

The minimum ordered flow is:

1. Validate the action request, authority grant, and material-evidence set against the applicable schemas.
2. Evaluate the pre-execution gate.
3. If the decision is **deny**, do not apply the proposed state change and create a denied or escalated event record.
4. If the decision is **allow**, execute only the authorized action and observe the outcome.
5. Create the receipt from the request, gate result, observed action state, and material evidence.
6. Compute integrity protection over the canonical receipt payload.
7. Persist the machine receipt before presenting a success state to the caller.
8. Render any human version only from the persisted machine receipt.

An executor MUST NOT treat receipt creation as optional post-processing. For actions governed by this profile, inability to create a durable receipt is a gate failure.

5. Data contracts and schema use

The **schemas** directory contains JSON Schema Draft 2020-12 documents for:

Schema	Purpose
<code>authority-grant.schema.json</code>	Bounded authorization and delegation policy.
<code>action-request.schema.json</code>	Proposed action and declared consequence context.
<code>material-evidence.schema.json</code>	Evidence availability, provenance, materiality, and freshness.
<code>trust-receipt.schema.json</code>	Canonical event receipt and integrity metadata.
<code>conformance-assessment.schema.json</code>	R1–R12 assessment result and operating mode.

Implementations **MUST** reject malformed or schema-incompatible input before policy evaluation. Structural schema validity does not establish authorization. Cross-field and temporal invariants in this document remain **REQUIRED** even where they are not fully expressible in the schema artifacts.

The reference validator implements only the Draft 2020-12 keywords used by the bundled schemas. It is dependency-free and is not a general-purpose JSON Schema implementation. Production systems **SHOULD** use a maintained Draft 2020-12 validator and **MUST** retain the runtime rules described below.

Unknown properties are rejected in the core schemas. An extension profile **MUST** use a versioned extension object or a new schema identifier rather than silently adding fields to the core record.

The v0.1.1 action-request schema permits explicit `action.reversible`, `consequence.protected_third_party_information`, and `human_review.evidence_viewed` fields. Implementations **MUST NOT** infer reversibility solely from persistence or infer evidence viewed from evidence availability. An approved review in the v0.1.1 reference gate is denied unless every required evidence identifier is explicitly recorded as viewed and every recorded identifier exists in the available evidence set. Implementations **MUST NOT** assume the absence of protected third-party information when the deployment has contrary evidence.

6. Trigger and consequence classification

6.1 Receipt trigger

The receipt profile is triggered when any of the following is true:

- The action creates or changes persistent external state.
- The action changes access, eligibility, identity, financial position, contractual position, communication, or another material status.
- The action acts on behalf of a person or organization outside the immediate conversational response.
- The action delegates authority or relies on a delegated action.
- The action affects an identifiable third party.
- The action requires notice, review, reversal, restoration, or contestability.

An implementation **MAY** issue receipts for lower-impact events. It **MUST NOT** suppress a receipt merely because the execution failed or was denied. Attempts and exceptions can be material evidence of how authority was enforced.

6.2 Consequence classes

Class	Operational meaning	Default consequence
C0	Transient or administrative event with no material external consequence and no persistent change.	Ordinary consequence
C1	Limited, reversible effect with low material impact and a clear correction path.	Bounded consequence
C2	Material effect on an affected party, persistent state, important access or status, or meaningful reliance.	Confirmed consequence
C3	Severe, difficult-to-reverse, rights-affecting, safety-critical, high-value, or comparably consequential effect.	Explicit dependency

The requester **MUST** declare a consequence class. The accountable organization **MUST** document classification criteria and **MUST** detect material under-classification. Where classification is ambiguous between two classes, the implementation **SHOULD** select the higher class or require accountable review.

7. Pre-execution gate

7.1 Inputs

The gate consumes exactly one action request, one authority grant, and one material-evidence set. The gate **MUST** evaluate immutable snapshots or versioned records. A caller **MUST NOT** alter these inputs during evaluation.

7.2 Deterministic rules

The reference decision is **allow** only when every applicable condition below passes. Otherwise, the decision is **deny**.

Code	Failure condition
GRANT_NOT_ACTIVE	Grant status is not active .
GRANT_NOT_YET_VALID	Request time precedes granted_at .
GRANT_EXPIRED	Request time is at or after expires_at .
GRANTEE_MISMATCH	Agent or operator identity differs from the grantee.
ACTION_OUT_OF_SCOPE	Action is absent from the allowlist or present in the exclusion list.
TARGET_OUT_OF_SCOPE	Target is not permitted by the grant.
CONSEQUENCE_EXCEEDS_GRANT	Declared consequence exceeds the grant ceiling.
CONFIRMATION_REQUIRED	Required user confirmation is absent.
HUMAN_REVIEW_REQUIRED	Required independent approval is absent.
DELEGATION_PROHIBITED	Delegation occurs when the grant prohibits it.
DELEGATION_DEPTH_EXCEEDED	Delegation depth exceeds the maximum.
DELEGATE_NOT_PERMITTED	A recipient is not in the permitted-recipient set.
REQUIRED_EVIDENCE_UNAVAILABLE	Required material evidence is not available.
REQUIRED_EVIDENCE_NOT_CURRENT	Required material evidence is stale or of unknown freshness.
RECEIPT_SERVICE_UNAVAILABLE	The system cannot produce the required durable receipt.
REMEDY_SERVICE_UNAVAILABLE	A C2 or C3 action lacks the required remedy capability.

The gate **MUST** return all detected failures, not only the first, so that correction and review are efficient. The executor **MUST** treat any non-empty failure set as a denial. An implementation **MUST NOT** fall back from denied agentic execution to a materially equivalent unrecorded automated path.

7.3 Temporal semantics

All date-times **MUST** include an offset. The grant interval is half-open: **granted_at** ≤ **requested_at** < **expires_at**. Implementations **SHOULD** use synchronized clocks, record clock source and uncertainty where relevant, and reject times whose reliability is insufficient for the consequence class.

7.4 Failure containment

For a denied action, **after_state** **MUST** equal **before_state** unless the receipt explicitly records a partial execution exception. A deployment that can partially mutate state **MUST** define transactional rollback or compensating-action behavior. It **MUST** not record a clean denial if an external change actually occurred.

If the primary receipt service is unavailable, the action **MUST** be denied and the gate decision **MUST** be written to an independent fail-safe event channel or caller-owned audit boundary. The reference package models this condition and can produce a local denial record; production deployment profiles **MUST** identify the independent failure channel.

8. Receipt semantics

8.1 Required record

A receipt conforming to this profile **MUST** identify:

- The receipt version, unique receipt identifier, event time, creation time, schema identifier, issuer, and verification endpoint.
- The agent, operator, deployment where available, and accountable organization.
- The action type, verb, target, status, persistence, and before and after states.
- The authority grant, granting party, scope, validity interval, confirmation status, and validation result.
- The objective and hard constraints governing the action.
- Each material input, provenance reference, materiality, availability status, and freshness.
- Delegation recipients, task, passed authority, depth, and returned evidence.
- Human-review status, authority to intervene, and evidence identifiers explicitly recorded as viewed by the reviewer where review occurred.

- Consequence class, affected-party state, notice requirement, persistence, and protected-third-party-information flag where known.
- Remedy availability, reversibility, contestability, accountable review owner, status route, and time limit.
- Retention basis, routine-access expiry, access roles, prohibited secondary uses, and excluded content.
- Relationships to earlier receipts where the event follows, supersedes, reverses, delegates from, or remedies an earlier event.
- Integrity method, canonicalization profile, digest value, and recorded verification state.

The receipt **MUST** record the observed event, not merely the requested event. The reference pipeline simulates successful execution by adopting the proposed state after an **allow** decision; a real executor **MUST** substitute its observed result.

8.2 Event status

The core action status vocabulary is:

Status	Meaning
attempted	Execution began or was requested, but no terminal result is yet asserted.
executed	The declared state transition was observed.
denied	The pre-execution or execution boundary rejected the action.
delegated	Bounded authority and task were passed to another actor.
escalated	The action was routed to accountable human or higher-authority review.
reversed	A later event restored or compensated for an earlier event.
expired	Authority, action opportunity, or pending execution elapsed without execution.

Material state transitions **MUST** be represented by a new receipt rather than overwriting an earlier receipt. A reversal **MUST** reference the earlier receipt using a **reverses** relationship. A corrected record **SHOULD** supersede rather than erase the original unless deletion is legally or ethically required; the deployment profile **MUST** state how integrity and affected-party access are preserved when deletion is required.

8.3 Identifier and ordering rules

Receipt identifiers **MUST** be unique within the issuer namespace and **SHOULD** be globally unique. Creation time is not sufficient as a unique identifier. The v0.1.1 reference implementation generates a deterministic 256-bit SHA-256 suffix over the complete request, grant, evidence-set, and issuer objects plus terminal status and creation time. Identical canonical inputs intentionally produce the same identifier; deployments remain responsible for unique request identifiers and replay or idempotency policy. A production deployment **MAY** replace this construction with an issuer-controlled sequence, UUID, ULID, or signed event identifier, provided uniqueness and verifier behavior are documented. Ordering across distributed systems **SHOULD** rely on explicit event relationships and issuer sequence controls where needed, not solely on wall-clock timestamps.

9. Canonicalization and integrity

9.1 Reference digest profile

The reference implementation uses **sha-256-digest-demo** with canonicalization profile **ei-canonical-json-no-floats-v0.1**.

The canonicalizer's shared Python/browser domain:

1. Rejects all floating-point values.
2. Rejects integers outside the JavaScript safe-integer range.
3. Rejects unpaired Unicode surrogates and non-string object keys.
4. Encodes JSON as UTF-8.
5. Sorts object keys lexicographically by Unicode code point.
6. Emits no insignificant whitespace.
7. Preserves array order.
8. Computes the digest over the entire receipt except the **integrity.digest** field.

The `integrity_verification_status` is included in the digest payload. Changing it invalidates the digest. The output format is `sha256:` followed by 64 lowercase hexadecimal characters. The v0.1.1 receipt schema accepts only `sha-256-digest-demo`; it does not reserve a partly specified digital-signature value. A signature method requires a later versioned profile that defines its complete envelope and verification policy.

This restricted profile is deterministic for the supported data types accepted by the bundled schemas. Open state objects **MUST** also remain inside the restricted domain. It is aligned in purpose with RFC 8785 but is not a complete implementation of the JSON Canonicalization Scheme. Implementations **MUST NOT** label it JCS or claim RFC 8785 conformance. The bundled browser parity vectors show matching digests for identical fixtures; they do not establish parity for values outside this domain.

9.2 Verification

A verifier **MUST** reconstruct the canonical payload from the receipt, recompute the digest, and use constant-time comparison where exposure warrants it. A digest mismatch **MUST** produce a failed verification result and **MUST NOT** be silently repaired.

A plain digest detects accidental or malicious alteration only when the verifier has a trusted expected digest or trusted receipt channel. It does not authenticate the issuer. A production profile requiring issuer authentication **SHOULD** use a recognized digital-signature or data-integrity mechanism, protected keys, key identifiers, rotation, revocation, and a verification policy appropriate to its risk.

9.3 Storage

Receipt storage **SHOULD** be append-oriented and **SHOULD** expose detectable modification or deletion. Backups, replication, access logs, and recovery behavior **SHOULD** be proportionate to consequence and retention requirements. Integrity protection does not remove the need for authorization, encryption, minimization, or operational monitoring.

10. Human and machine consistency

The machine receipt is the canonical source. A human-readable receipt **MUST** be rendered from that exact record after integrity material is added. A system **MUST NOT** maintain a separate hand-authored narrative containing independently supplied status, authority, consequence, evidence, or remedy facts.

At minimum, the human projection **MUST** show the receipt identifier, event time, action and status, target, accountable organization, grant and authority status, consequence class, affected-party and notice flags, material-evidence status, human-review state, remedy route, and integrity value.

A renderer **MAY** simplify terminology, localize language, or progressively disclose detail. It **MUST** preserve the semantic value of enumerated fields. If localization is uncertain, the original machine value **SHOULD** remain visible. The reference test harness checks that the receipt identifier, action status, and digest in the human output exactly match the machine record.

11. Privacy, minimization, and excluded content

A receipt **MUST** contain enough evidence to support verification and remedy, but **MUST NOT** become a general-purpose surveillance record. Implementations **MUST** document purpose, access roles, routine-access expiry, retention basis, deletion behavior, and prohibited secondary uses.

The core schema requires `chain_of_thought_excluded: true`. Hidden model reasoning, private scratch work, and unrestricted prompt or memory dumps **MUST NOT** be stored as receipt content. An implementation **MAY** record concise stated reasons, policy identifiers, evidence references, input and output hashes, and externally reviewable decision facts.

Secrets, authentication tokens, private keys, unrelated personal information, full protected records when a reference will suffice, and protected third-party content **MUST NOT** be included. Evidence references **MUST** themselves be access-controlled and **SHOULD** reveal no more information than necessary.

Routine operational access **SHOULD** expire before archival or legally required retention ends. After routine access expires, access **SHOULD** require an elevated, recorded purpose such as dispute review, security investigation, or legal obligation.

12. Notice, remedy, and affected-party access

When `notice_required` is true, the deployment MUST provide timely notice through an accessible channel unless a documented lawful exception applies. Notice SHOULD identify the event, effect, accountable organization, verification route, and remedy route without exposing protected third-party information.

For C2 and C3 actions, remedy capability MUST be operational before execution. A static policy page is insufficient if no accountable reviewer can act. The receipt MUST name a review owner, route to status or contest, and state the applicable time limit. Reversibility is not equivalent to persistence: a persistent change can be reversible or nonreversible, and a nonpersistent action can still require remedy. Where an action is reversible, the deployment SHOULD preserve the minimum before-state or restoration evidence needed to reverse it safely.

Remedy events SHOULD produce their own receipts and reference the original using `remedies` or `reverses`. A reviewer MUST be able to distinguish the original decision, the contested assertion, the review evidence, and the final disposition.

13. Delegation

Delegation does not expand authority. Each delegated action MUST remain within the original objective, action scope, target scope, consequence ceiling, validity interval, and delegation depth. A recipient MUST be explicitly permitted by the grant. The delegator MUST record the task, authority passed, recipient, depth, and whether evidence returned.

If delegated evidence is missing, stale, disputed, or unavailable and is material to execution, the parent action MUST fail closed. A deployment MUST NOT convert missing delegated evidence into an assumed success.

14. R1–R12 conformance profile

14.1 Requirements

ID	Requirement	Minimum evidence question
R1	Durable receipt by event state	Can each attempted, executed, denied, delegated, escalated, reversed, or exp
R2	Actor and accountability identity	Are agent, operator, and accountable organization unambiguous and verifiab
R3	Action and state change	Does the record distinguish requested, observed, before, and after state?
R4	Authority bound to action	Is a valid grant tied to action, target, consequence, time, and confirmation?
R5	Material inputs and provenance	Are decision-changing inputs identified with provenance, status, materiality,
R6	Delegation and human oversight	Are delegation boundaries and meaningful human intervention recorded?
R7	Consequence and affected-party status	Is consequence class justified and are affected-party and notice states explici
R8	Remedy and accountable review	Is there an operational route to contest, review, reverse, or restore?
R9	Retention, access, and secondary use	Are purpose, access, expiry, deletion, and prohibited uses defined?
R10	Protected and excluded content	Are secrets, unnecessary protected data, and chain-of-thought excluded?
R11	Cross-format consistency and verification	Do human and machine records agree, and can integrity be checked?
R12	Safe failure when evidence is unavailable	Does missing authority, evidence, receipt, or remedy capability block executi

14.2 Per-requirement values

Each requirement receives exactly one result:

- **pass**: The requirement is fully satisfied for the declared boundary, and material verification evidence is available.
- **partial**: The requirement is materially implemented but has a bounded gap, with an owner, due date, verification method, and operating restriction where agentic operation continues.
- **fail**: The requirement is absent, materially ineffective, contradicted by evidence, or cannot be verified because material evidence is unavailable.

A claimed **pass** or **partial** with `evidence_available: false` MUST be normalized to **fail**. Assessment evidence SHOULD identify versions, time boundaries, systems, samples, and verification methods.

14.3 Overall decision

The decision model is noncompensatory:

```
if any requirement == fail:
    overall_decision = does_not_conform
elif any requirement == partial:
    overall_decision = conditionally_conforms
else:
    overall_decision = conforms
```

No score, average, weight, or maturity level may override a failure under this candidate profile.

14.4 Execution mode

Condition	Required execution mode
Any failure	Disabled or manual control
C3 with any partial	Manual control
C2 with a partial in R1, R2, R4, R8, R10, R11, or R12	Manual control
Other conditional conformance	Restricted agentic operation
Full conformance	Approved boundary

The corresponding machine values are `disabled_or_manual_control`, `manual_control`, `restricted_agentic_with_time_boundaries`, and `approved_boundary`.

`approved_boundary` means only the assessed system, version, use, consequence boundary, and operating environment. It is not unrestricted approval. A material system, policy, model, authority, consequence, or remedy change SHOULD trigger reassessment.

15. Interfaces

The reference command-line interface is organized into three groups:

```
trust-receipt gate evaluate
trust-receipt receipt create
trust-receipt receipt verify
trust-receipt receipt render
trust-receipt conformance assess
trust-receipt conformance run
```

CLI success returns status code 0. A valid but denied gate or failed receipt verification returns 2. Invalid input or schema failure returns 3. A conformance-vector run with one or more mismatches returns 1.

A network service MAY expose equivalent operations. It SHOULD return an RFC 9457 problem-details response for API errors and MUST NOT expose protected evidence in diagnostic detail. A service MUST preserve the same deterministic gate and conformance semantics as the command-line implementation.

16. Error handling and observability

Errors are classified as:

Class	Examples	Required behavior
Input	Invalid JSON, schema mismatch, unsupported version	Reject before policy evaluation; return bounded field-level error
Authority	Expired grant, scope mismatch, missing confirmation	Deny; record the applicable gate code; do not mutate state
Evidence	Missing, disputed, stale, unknown	Deny when material; preserve evidence status without i
Dependency	Receipt or remedy service unavailable	Deny governed action; use the documented fail-safe eve
Execution	Timeout, partial mutation, target error	Record observed state; roll back or compensate where p

Class	Examples	Required behavior
Integrity	Digest verification failure	Mark verification failed; do not overwrite the source record

Operational logs SHOULD use receipt identifiers and bounded error codes, not full sensitive receipt bodies. Metrics SHOULD include gate denials by code, receipt persistence failures, verification failures, remedy availability, reversal latency, and unresolved partial-conformance deadlines.

17. Security considerations

Threats include forged authority, replayed confirmation, altered consequence classification, confused-deputy delegation, stale evidence, target substitution, race conditions between gate and execution, partial mutation, forged receipts, unauthorized disclosure, deletion, replay, and a human narrative that diverges from the machine record.

Deployments SHOULD bind grants and confirmations to subject, action, target, nonce or request identifier, and validity interval. Gate evaluation and execution SHOULD be atomic or protected by a version check on the target state. High-consequence systems SHOULD use issuer-authenticated signatures, protected key material, independent verification, immutable or transparency-capable storage, and separation between action authority and conformance assessment.

Receipt verification proves only what the selected integrity method establishes. It does not prove the truth of evidence, correctness of consequence classification, adequacy of remedy, or legitimacy of authority. Those claims require governance and, where appropriate, independent assurance.

18. Reference test suite

The bundled suite contains structural, policy, integrity, projection, and conformance checks. The normative conformance vectors are:

Vector	Expected result
T01	All requirements pass at C0 → conforms / approved boundary.
T02	Noncritical partial at C1 → conditionally conforms / restricted agentic.
T03	Any failure at C1 → does not conform / disabled or manual.
T04	Any partial at C3 → conditionally conforms / manual.
T05	Critical partial at C2 → conditionally conforms / manual.
T06	Noncritical partial at C2 → conditionally conforms / restricted agentic.
T07	Unavailable evidence overrides pass → does not conform / disabled or manual.
T08	All requirements pass at C3 → conforms / approved boundary.
T09	Any failure at C3 → does not conform / disabled or manual.

The v0.1.1 automated tests verify that valid inputs pass the gate; malformed schema inputs are rejected; out-of-scope targets, expiration boundaries, missing confirmation, missing human review, incomplete review-evidence declarations, unpermitted delegation, stale required evidence, unsafe generated-link bases, unavailable receipt service, and unavailable C2 remedy fail closed; denied receipts record no state change; receipts validate against the schema; receipt identifiers bind the complete input preimage; explicit reversibility and protected-third-party-information fields are projected; unsafe canonicalization values are rejected; action or verification-status tampering invalidates the digest; and the CLI creates and verifies a receipt end to end. Separate Node.js vectors reproduce the Python canonicalization and digest results for identical bundled values.

Passing the included tests demonstrates consistency with the included implementation and vectors only. It is not third-party certification or external audit evidence.

19. Versioning and extension

Core schema and profile versions use explicit identifiers. The v0.1.1 patch advances the receipt and action-request schema identifiers to 0.1.1 because it corrects receipt integrity semantics and action-request inputs. Unchanged

authority, evidence, assessment, and conformance-profile contracts retain their v0.1 identifiers. A backward-incompatible receipt field, integrity semantic, or conformance-decision change requires a new core version. A verifier **MUST** reject unknown core versions unless an explicitly configured compatibility profile applies.

Extensions **MUST** define their namespace, schema, semantic effect, privacy impact, rendering behavior, and whether they are included in integrity protection. An extension **MUST NOT** weaken a core **MUST** while claiming conformance to the unchanged profile.

Future candidate work may define signature suites, issuer discovery, receipt status and revocation, selective disclosure, encrypted evidence references, sector-specific consequence profiles, transparency logs, remedy workflow events, and formal protocol APIs.

20. References

- JSON Schema, Draft 2020-12.
- RFC 8785, JSON Canonicalization Scheme.
- RFC 9457, Problem Details for HTTP APIs.
- W3C, Verifiable Credentials Data Model 2.0.
- W3C, Verifiable Credential Data Integrity 1.0.
- W3C, PROV-O: The PROV Ontology.
- NIST NCCoE, Software and AI Agent Identity and Authorization.

Implementation note: The source of truth for this candidate is the versioned Markdown specification, JSON schemas, decision profile, executable reference package, and bundled conformance vectors. A rendered PDF is a convenience copy and **MUST** identify the same candidate version.